

Votre parcours d'apprentissage

Module 1 : Configuration de l'environnement (25 minutes)

Pour commencer, vous avez préparé votre environnement de développement, installé Python, créé un environnement virtuel, installé ADK et configuré les clés API. Vous avez appris la procédure de configuration correcte et vérifié que tout fonctionnait en créant et en exécutant votre premier agent avec `adk create` et `adk web`.

Concepts clés maîtrisés :

- Installer ADK avec `pip install google-adk`
- Créer des environnements virtuels pour les projets Python
- Configurer des fichiers `.env` avec `GOOGLE_API_KEY` après avoir exécuté `adk create`
- Comprendre la structure de fichiers générée : `agent.py`, `__init__.py` et `.env`
- Vérifier la configuration avec `adk web`

Module 2 : Comprendre votre premier agent (20 minutes)

Vous avez découvert les quatre paramètres clés qui définissent chaque agent : `model`, `name`, `description` et `instruction`. Vous avez transformé l'agent par défaut en professeur de mathématiques spécialisé en personnalisant son instruction. Vous avez également découvert la différence entre la description (pour les autres agents) et l'instruction (pour le comportement de cet agent).

Concepts clés maîtrisés :

- Quatre paramètres clés : "model", "name", "description" et "instruction"
- Convention de dénomination de la variable `root_agent` que les outils ADK recherchent
- Rédiger des instructions efficaces qui définissent la personnalité et le comportement de l'agent
- Savoir quand utiliser le paramètre `description` plutôt que le paramètre `instruction`
- Tester des agents avec `adk web`

Module 3 : Autres façons d'exécuter votre agent (15 minutes)

Vous avez découvert trois façons d'exécuter des agents en dehors de l'interface Web : exécution depuis le terminal avec `adk run`, déploiement de l'API REST avec `adk api_server` et exécution programmatique dans du code Python. Chaque méthode répond à des besoins différents, allant des tests rapides à l'intégration en production.

Concepts clés maîtrisés :

- `adk run` pour effectuer des tests rapides basés sur le terminal
- `adk api_server` pour déployer des agents en tant qu'API REST
- Exécution programmatique à l'aide de Runner, de sessions et de boucles d'événements asynchrones

- Quand utiliser chaque méthode d'exécution
- Exécution à partir du répertoire d'agent ou du répertoire parent

Module 4 : Configuration d'agent avec YAML (15 minutes)

Vous avez exploré une deuxième façon de définir des agents, en utilisant des fichiers de configuration YAML au lieu de code Python. Vous avez appris que `adk create --type=config` génère un fichier `root_agent.yaml`, ce qui permet aux non-programmeurs de créer des agents. Vous avez découvert quand utiliser YAML (agents simples et prototypage rapide) plutôt que du code Python (systèmes multi-agents complexes et outils personnalisés).

Concepts clés maîtrisés :

- Créer des agents basés sur YAML en utilisant `adk create --type=config`
- Écrire des configurations d'agents au format YAML
- Comprendre que la configuration d'agent YAML est uniquement compatible avec Python
- Savoir quand utiliser YAML plutôt que Python pour définir un agent
- Exécuter des agents YAML de la même manière que des agents Python

Actions possibles actuellement

Grâce aux compétences acquises dans ce cours, vous pouvez :

1. **créer des agents personnalisés** pour n'importe quel usage (professeurs, assistants ou rédacteurs créatifs) ;
2. **exécuter les agents de différentes manières** en fonction de vos besoins (UI Web, terminal, API ou de manière programmatique) ;
3. **définir des agents en Python ou en YAML** selon vos préférences et votre cas d'utilisation ;
4. **créer des applications basées sur des agents** à l'aide du modèle Runner avec des sessions.

Points clés à retenir

Pour la suite, rappelez-vous ceci :

1. **Il y a quatre paramètres clés.** Chaque agent a besoin d'un modèle, d'un nom, d'une description et d'une instruction.
2. **L'instruction est essentielle.** Ce paramètre définit le comportement et les interactions de votre agent.
3. **Il existe plusieurs méthodes d'exécution** : choisissez `adk web` pour le développement, `adk run` pour les tests rapides, `adk api_server` pour les API ou l'exécution programmatique pour les applications personnalisées.
4. **Python ou YAML** : utilisez YAML pour sa simplicité et son accessibilité, et Python pour les systèmes complexes.

5. **La configuration de l'environnement est importante.** Configurez toujours `.env` avec `GOOGLE_API_KEY` après avoir exécuté `adk create`.

Communauté et support

Principales références

-  [Documentation ADK](#) – Documentation de référence officielle sur ADK
-  [Guide de démarrage rapide Python pour ADK](#) – Guide de démarrage
-  [Documentation sur la configuration d'agent](#) – Documentation de référence sur la configuration YAML

Ressources

De Google

- [Atelier de programmation "Créer des agents IA avec ADK : les bases"](#) : atelier de programmation pratique pour acquérir des compétences de base en création d'agents

De la communauté

- [Vidéo "Tutoriel Google Cloud Agent Development Kit | Créez votre premier agent ADK"](#) : guide pour les débutants sur la création d'agents
- [Vidéo "Formation de la communauté Google AI : Premiers pas avec Google Agent Development Kit"](#) : formation complète sur Google ADK
- [Article "Chapter 1 – Introduction to AI agents and the Agent Development Kit \(ADK\)"](#) : analyse approfondie de la création d'agents IA
- [Cours "Hands-on AI: Building agents with the Google Agent Development Toolkit \(ADK\)"](#) : cours LinkedIn Learning sur la création pratique d'agents IA
- [Build your first AI Agent with ADK – Agent Development Kit by Google](#) : tutoriel pas à pas sur la configuration de l'environnement et la création de votre premier agent conversationnel
- [Your First ADK Agent: Building a Google Tasks To-Do Manager](#) : exemple pratique de création d'un agent réel avec authentification OAuth
- [Building and deploying a Python ADK agent from the Google Cloud Shell](#) : article écrit explorant une configuration d'environnement alternative à l'aide de Cloud Shell au lieu d'une installation locale
- [Use your ADK AI agent in a UI](#) : article expliquant comment intégrer votre agent dans une UI Web avec des exemples de code pratiques
- [Building and deploying AI agents in minutes with Google's ADK – Part 1](#) : tutoriel complet allant de la configuration au déploiement, avec plusieurs méthodes d'exécution

Des questions ? Posez-les sur le [forum de la communauté](#).

Mémento

Commandes essentielles

```
# Configuration de l'environnement
python3 -m venv adk-env          # Créer un environnement virtuel
source adk-env/bin/activate      # Activer (macOS/Linux)
adk-env\Scripts\activate         # Activer (Windows)
pip install google-adk          # Installer ADK

# Créer des agents
adk create my_agent              # Créer un agent basé sur Python
adk create --type=config my_agent # Créer un agent basé sur YAML

# Exécuter des agents
adk web                          # Interface Web (depuis le répertoire d'agent)
adk web my_agent                 # Interface Web (depuis le répertoire parent)
adk run                          # Exécution depuis le terminal
adk api_server                   # Serveur d'API REST
```

Modèle d'agent basé sur Python

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Aide les élèves à apprendre l'algèbre en les guidant à travers les
étapes de résolution de problèmes.',
    instruction="""Tu es un professeur d'algèbre patient et encourageant.

    Ton approche pédagogique :
    1. Lorsqu'un élève pose une question, commence par identifier ce qui le met en
difficulté
    2. Décompose le problème en étapes plus petites et gérables
    3. Guide l'élève pour qu'il découvre la réponse plutôt que de la lui donner
directement
    4. Encourage positivement les efforts et les progrès de l'élève
    5. Utilise un langage simple et évite le jargon

    Garde toujours un ton bienveillant et patient."""
)
```

Modèle d'agent basé sur YAML

```
# root_agent.yaml
name: math_tutor_agent
model: gemini-2.5-flash
description: Aide les élèves à apprendre l'algèbre en les guidant à travers les étapes
de résolution de problèmes.
```

```
instruction: |
    Tu es un professeur d'algèbre patient et encourageant.

    Ton approche pédagogique :
    1. Lorsqu'un élève pose une question, commence par identifier ce qui le met en
difficulté
    2. Décompose le problème en étapes plus petites et gérables
    3. Guide l'élève pour qu'il découvre la réponse plutôt que de la lui donner
directement
    4. Encourage positivement les efforts et les progrès de l'élève
    5. Utilise un langage simple et évite le jargon

    Garde toujours un ton bienveillant et patient.
```

Modèle d'exécution programmatique

```
import asyncio
from google.adk.agents.llm_agent import Agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Créer un agent
agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor',
    instruction='Tu es un professeur de mathématiques patient.'
)

# Configurer Runner
session_service = InMemorySessionService()
runner = Runner(
    agent=agent,
    app_name='my_app',
    session_service=session_service
)

# Exécuter l'agent
async def run_agent():
    session = await session_service.create_session(
        app_name='my_app',
        user_id='user_1',
        session_id='session_1'
    )

    user_message = Content(
        role='user',
        parts=[Part(text='Comment résoudre l\'équation  $2x + 5 = 13$  ?')]
    )

    async for event in runner.run_async(
        user_id='user_1',
        session_id='session_1',
        new_message=user_message
    ):
```

```
    ):
        if event.is_final_response() and event.content and event.content.parts:
            print(event.content.parts[0].text)

# Exécuter
asyncio.run(run_agent())
```